ORACLE SQL + PL/SQL LAB CHEATSHEET

Exam revision reference for a Library Management System

SQL, joins, subqueries, views, TCL/DCL, PL/SQL blocks, conditions, loops, cursors, procedures, functions and triggers

Important: Examples assume common library columns such as `book.book_id`, `book.title`, `staff.salary`, `member.full_name`, and `loan.member_id`. Check your actual schema and adjust column names/relationships. SQL*Plus/SQL Developer script examples use `/` to execute a completed PL/SQL block or stored program.

1 Basic SQL

1.1 SELECT template

```
SELECT column1, column2
FROM table_name
WHERE condition
GROUP BY column1
HAVING aggregate_condition
ORDER BY column1 ASC;
```

Clauses are optional except `SELECT` and `FROM` in a normal query. Logical purpose: choose columns, choose tables, filter rows, group, filter groups, sort.

1.2 Filtering and operators

```
WHERE price > 500
WHERE status = 'ACTIVE'
WHERE price BETWEEN 200 AND 800
WHERE category IN ('Fiction', 'Science')
WHERE title LIKE 'The%'      -- starts with The
WHERE title LIKE '%SQL%'     -- contains SQL
WHERE publisher_id IS NULL
WHERE status <> 'INACTIVE'
WHERE price > 300 AND category = 'Science'
WHERE status = 'ACTIVE' OR status = 'PENDING'
```

Remember: use single quotes for text; use `IS NULL` / `IS NOT NULL`; `%` matches any sequence and `_` matches one character.

1.3 Aggregate functions

```
SELECT COUNT(*) FROM book;
SELECT SUM(price), AVG(price), MIN(price), MAX(price) FROM book;
SELECT category, COUNT(*) AS book_count
FROM book
GROUP BY category
HAVING COUNT(*) > 2;
```

`WHERE` filters rows before grouping; `HAVING` filters groups after aggregation.

2 JOINs

2.1 INNER JOIN: matching rows only

```
SELECT b.title, p.publisher_name
FROM book b
INNER JOIN publisher p ON b.publisher_id = p.publisher_id;
```

2.2 LEFT JOIN: every left row, matches from right if present

```
SELECT l.branch_name, m.full_name
FROM library_branch l
LEFT JOIN member m ON l.branch_id = m.branch_id;
```

2.3 RIGHT JOIN: every right row

```
SELECT l.branch_name, m.full_name
FROM member m
RIGHT JOIN library_branch l ON m.branch_id = l.branch_id;
```

2.4 FULL OUTER JOIN: all rows from both sides

```
SELECT l.branch_name, m.full_name
FROM library_branch l
FULL OUTER JOIN member m ON l.branch_id = m.branch_id;
```

2.5 NATURAL JOIN

```
SELECT title, publisher_name
FROM book NATURAL JOIN publisher;
```

Oracle joins columns with matching names automatically. Use only when shared column names are intentional; explicit `JOIN ... ON` is safer.

2.6 Multiple-table JOIN pattern

```
SELECT m.full_name, b.title, l.issue_date, l.status
FROM member m
JOIN loan l ON m.member_id = l.member_id
JOIN book_copy c ON l.copy_id = c.copy_id
JOIN book b ON c.book_id = b.book_id;
```

Join mental model: follow the foreign-key path between tables. For a LEFT JOIN, the table whose every row must remain goes on the left. Use distinct aliases for columns with similar names.

3 Subqueries

A subquery is a query inside another SQL statement.

3.1 Single-value subquery

```
-- Books priced above average
SELECT title, price FROM book
WHERE price > (SELECT AVG(price) FROM book);

-- Staff member(s) with lowest salary
SELECT * FROM staff
WHERE salary = (SELECT MIN(salary) FROM staff);

-- Staff who joined after staff ID 502
SELECT * FROM staff
WHERE join_date > (SELECT join_date FROM staff WHERE staff_id = 502);
```

3.2 Multiple-value subquery with IN

```
-- Books published by publishers in Kochi
SELECT title FROM book
WHERE publisher_id IN
  (SELECT publisher_id FROM publisher WHERE city = 'Kochi');
```

3.3 Same branch as member 1001

```
SELECT * FROM member
WHERE branch_id = (SELECT branch_id FROM member WHERE member_id = 1001)
  AND member_id <> 1001;
```

3.4 EXISTS and correlated subqueries

```sql
-- Staff with at least one loan (adjust relationship if schema differs)
SELECT s.* FROM staff s
WHERE EXISTS (SELECT 1 FROM loan l WHERE l.staff_id = s.staff_id);

-- Staff earning more than the average salary in their branch
SELECT s.staff_id, s.salary FROM staff s
WHERE s.salary > (SELECT AVG(s2.salary) FROM staff s2
                  WHERE s2.branch_id = s.branch_id);
```

Operator guide: use = or > when the subquery returns one value; use `IN` for a list; use `EXISTS` to test whether matching rows exist. `ANY` and `ALL` compare against a set of values.

3.5 Second-highest distinct salary

```sql
SELECT MAX(salary) AS second_highest_salary
FROM staff
WHERE salary < (SELECT MAX(salary) FROM staff);
```

This returns the second-highest distinct salary; it returns NULL if there is no lower salary.

4 Views

```sql
CREATE VIEW book_details AS
SELECT book_id, title, publisher_id, price FROM book;

SELECT * FROM book_details;
SELECT title, price FROM book_details WHERE price > 500;

CREATE OR REPLACE VIEW book_details AS
SELECT book_id, title, price FROM book;

DROP VIEW book_details;
```

A view is a named query (virtual table), not normally a separate copy of the data. DML through a view depends on whether Oracle considers it updatable; views with joins, aggregates, grouping or DISTINCT commonly have restrictions.

```sql
CREATE VIEW book_publisher_details AS
SELECT b.title, p.publisher_name, b.price
FROM book b JOIN publisher p ON b.publisher_id = p.publisher_id;
```

5 TCL and DCL

5.1 TCL: transaction control

```sql
UPDATE book SET price = 550 WHERE book_id = 101;
SAVEPOINT after_first_update;
UPDATE book SET price = 900 WHERE book_id = 102;
ROLLBACK TO after_first_update; -- undo second update only
COMMIT;                         -- make remaining change permanent
```

`COMMIT` saves transaction changes; `ROLLBACK` undoes uncommitted changes; `SAVEPOINT` marks a point to roll back to. Oracle DDL statements generally cause implicit commits, so be careful when mixing DDL and DML.

5.2 DCL: permissions

```sql
GRANT SELECT ON book TO student_user;
GRANT SELECT, INSERT ON member TO student_user;
REVOKE SELECT ON book FROM student_user;
REVOKE INSERT ON member FROM student_user;
```

Use a real Oracle username in place of `student_user`; you need appropriate privileges to grant or revoke permissions.

6 PL/SQL block structure

```sql
SET SERVEROUTPUT ON;
DECLARE
    -- declarations (optional)
BEGIN
    -- executable statements (required)
EXCEPTION
```

```
    -- error handlers (optional)
END;
/
```

DECLARE holds variables/cursors; BEGIN starts execution; EXCEPTION handles runtime errors; END; closes the block. SET SERVEROUTPUT ON enables displayed output in supported script tools.

6.1 Print a message

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Welcome to the Library');
END;
/
```

6.2 Variables and assignment

```
DECLARE
    v_price NUMBER := 500;
    v_title VARCHAR2(100);
BEGIN
    v_price := v_price + 50;
    DBMS_OUTPUT.PUT_LINE('Price: ' || v_price);
END;
/
```

:= assigns a value; = compares values; || concatenates text and values. Prefixes such as v_ are a naming convention, not a requirement.

6.3 Read a column into a variable with SELECT INTO

```
DECLARE
    v_title book.title%TYPE;
    v_book_id book.book_id%TYPE := 101;
BEGIN
    SELECT title INTO v_title
    FROM book WHERE book_id = v_book_id;
    DBMS_OUTPUT.PUT_LINE('Title: ' || v_title);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Book not found');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one row returned');
END;
/
```

%TYPE takes the data type from a column/variable. A single-row SELECT INTO raises NO_DATA_FOUND for no rows and TOO_MANY_ROWS for multiple rows.

7 Conditional statements

7.1 IF

```
DECLARE
    v_price NUMBER := 750;
BEGIN
    IF v_price > 500 THEN
        DBMS_OUTPUT.PUT_LINE('Expensive book');
    END IF;
END;
/
```

7.2 IF / ELSE

```
IF v_price >= 500 THEN
    DBMS_OUTPUT.PUT_LINE('High price');
ELSE
    DBMS_OUTPUT.PUT_LINE('Low price');
END IF;
```

7.3 IF / ELSIF / ELSE

```
IF v_marks >= 80 THEN
    DBMS_OUTPUT.PUT_LINE('Distinction');
ELSIF v_marks >= 50 THEN
    DBMS_OUTPUT.PUT_LINE('Pass');
ELSE
    DBMS_OUTPUT.PUT_LINE('Fail');
END IF;
```

Use `ELSIF` in PL/SQL, not `ELSE IF`. Conditions use operators such as `=`, `<>`, `>`, `<`, `>=`, `<=`, `AND`, `OR`, `NOT`.

7.4 CASE statement

```
CASE v_status
    WHEN 'ACTIVE' THEN DBMS_OUTPUT.PUT_LINE('Member is active');
    WHEN 'SUSPENDED' THEN DBMS_OUTPUT.PUT_LINE('Member is suspended');
    ELSE DBMS_OUTPUT.PUT_LINE('Other status');
END CASE;
```

8 Loops

8.1 Basic LOOP with EXIT WHEN

```
DECLARE
    i NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 1;
        EXIT WHEN i > 5;
    END LOOP;
END;
/
```

A basic `LOOP` repeats until an `EXIT` or `EXIT WHEN` is reached. Make sure the exit condition can become true.

8.2 WHILE LOOP

```
DECLARE
    i NUMBER := 1;
BEGIN
    WHILE i <= 5 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 1;
    END LOOP;
END;
/
```

Checks the condition before each iteration; may run zero times.

8.3 FOR LOOP

```
BEGIN
    FOR i IN 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
/
```

The loop variable is automatically declared. Reverse order:

```
BEGIN
    FOR i IN REVERSE 1..5 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
/
```

Quick distinction: use `FOR` when the range is known; `WHILE` when repetition depends on a condition; basic `LOOP` when you want an explicit exit.

9 Cursors

A cursor processes rows returned by a query. An explicit cursor is useful when handling a result set row by row.

9.1 Explicit cursor template

```
DECLARE
    CURSOR c_books IS SELECT title FROM book;
    v_title book.title%TYPE;
BEGIN
    OPEN c_books;
    LOOP
        FETCH c_books INTO v_title;
        EXIT WHEN c_books%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_title);
    END LOOP;
    CLOSE c_books;
END;
/
```

Cursor lifecycle: declare, open, fetch, test `%NOTFOUND`, close.

9.2 Cursor FOR LOOP (shorter and safer for simple tasks)

```
BEGIN
    FOR r IN (SELECT title, price FROM book) LOOP
        DBMS_OUTPUT.PUT_LINE(r.title || ': ' || r.price);
    END LOOP;
END;
/
```

Oracle manages opening, fetching and closing the implicit cursor automatically.

10 Procedures

A procedure is a named stored program that performs an action. It can accept parameters.

10.1 Procedure that displays a book title

```
CREATE OR REPLACE PROCEDURE show_book_title (
    p_book_id IN book.book_id%TYPE
) AS
    v_title book.title%TYPE;
BEGIN
    SELECT title INTO v_title
    FROM book WHERE book_id = p_book_id;
    DBMS_OUTPUT.PUT_LINE(v_title);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Book not found');
END;
/

SET SERVEROUTPUT ON;
EXEC show_book_title(101);
```

Parameter modes: `IN` passes a value in; `OUT` returns a value; `IN OUT` passes a value in and can return a changed value.

10.2 Procedure to update a book price

```
CREATE OR REPLACE PROCEDURE update_book_price (
    p_book_id IN book.book_id%TYPE,
    p_price IN book.price%TYPE
) AS
BEGIN
    UPDATE book SET price = p_price
    WHERE book_id = p_book_id;
END;
/

EXEC update_book_price(101, 650);
-- COMMIT; -- execute separately if the change should be committed
```

11 Functions

A function returns a value using `RETURN`. It can be called from PL/SQL; functions used in SQL expressions have additional restrictions.

11.1 Function returning member name

```
CREATE OR REPLACE FUNCTION get_member_name (
    p_member_id IN member.member_id%TYPE
) RETURN VARCHAR2 AS
    v_name member.full_name%TYPE;
BEGIN
    SELECT full_name INTO v_name
    FROM member WHERE member_id = p_member_id;
    RETURN v_name;
END;
/
```

Call from PL/SQL:

```
DECLARE
    v_name VARCHAR2(200);
BEGIN
    v_name := get_member_name(1001);
    DBMS_OUTPUT.PUT_LINE(v_name);
END;
/
```

A function needs a return data type and a `RETURN` statement on the executed path. Add exception handling if missing IDs are possible.

11.2 Function returning total book count

```
CREATE OR REPLACE FUNCTION total_books
RETURN NUMBER AS
    v_total NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM book;
    RETURN v_total;
END;
/
```

12 Triggers

A trigger runs automatically when a specified database event occurs.

12.1 Trigger: uppercase a member's name before insert

```
CREATE OR REPLACE TRIGGER member_name_uppercase
BEFORE INSERT ON member
FOR EACH ROW
BEGIN
    :NEW.full_name := UPPER(:NEW.full_name);
END;
/
```

12.2 Trigger: reject negative book price

```
CREATE OR REPLACE TRIGGER check_book_price
BEFORE INSERT OR UPDATE OF price ON book
FOR EACH ROW
BEGIN
    IF :NEW.price < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Price cannot be negative');
    END IF;
END;
/
```

`:NEW` means the incoming/new row value; `:OLD` means the existing/old row value where applicable. `FOR EACH ROW` makes it a row-level trigger. Use `RAISE_APPLICATION_ERROR` to stop an invalid change with a custom error.

13 Exception handling quick reference

```
BEGIN
    -- statements
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No matching row');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Query returned multiple rows');
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/
```

Use `WHEN OTHERS` carefully; it should not hide errors silently. `SQLERRM` returns the error message.

14 Fast pattern-matching table

Question wording	**Likely construct**
Show rows matching in both tables	INNER JOIN
Show every row from one table, even without a match	LEFT or RIGHT JOIN
Above / below average	Subquery with AVG
Lowest / highest value	MIN / MAX, often in a subquery
Matches any IDs returned by another query	IN
At least one related row exists	EXISTS
Store a reusable query	VIEW
Make changes permanent / undo / partial undo	COMMIT / ROLLBACK / SAVEPOINT
Give / remove privileges	GRANT / REVOKE
Print a message or run simple logic once	Anonymous PL/SQL block
Repeat for a known range	FOR LOOP
Repeat while a condition is true	WHILE LOOP
Process rows returned by a query	Cursor
Reusable stored action	Procedure
Reusable stored calculation that returns a value	Function
Run automatically on insert/update/delete	Trigger

15 Common exam mistakes to avoid

1. Forgetting semicolons inside PL/SQL statements and after `END;`.
2. Using `=` instead of `:=` for PL/SQL assignment.
3. Forgetting `INTO` in a single-row PL/SQL `SELECT`.
4. Fetching a cursor without checking `%NOTFOUND` (can cause an extra/stale row to be processed).
5. Using `WHERE aggregate(...)` instead of grouping and using `HAVING` where appropriate.
6. Using `= NULL` instead of `IS NULL`.
7. Using a column from the wrong table after a JOIN; qualify ambiguous columns with aliases.
8. Assuming every view supports DML, or that every subquery returns one row.
9. Forgetting to execute stored-program definitions with `/` in script tools.
10. Assuming sample IDs such as 101 or 1001 exist in your own database; verify the data first.

Exam strategy: identify the required output, map the table relationships, choose the SQL/PLSQL construct, then verify column names and test the query.